Lab 5 实验报告

- 姓名：徐培森
- 学号：25300440003
- 邮箱：25300440003@m.fudan.edu.cn

一、预估修改与实际修改比对

1.1 人工预估

在动手实现前，根据 Lab5 需求文档，预估至少需要以下修改：

模块	预估修改点
新增游戏	新增 `chess` 包，实现 ChessBoard、ChessPiece、ChessGame 及行棋规则
胜负判定	ChessGame 中增加"王被吃掉即结束"的简化规则
输入解析	GameController 需新增 `m/move` 命令的解析分支
初始对局	MultiBoardManager 默认初始化由 3 局改为 4 局，新增 chess
界面适配	TerminalUI 的 `drawStatusInBox` 需新增 chess 分支，显示黑白双方回合
Demo 模式	GameController 增加自动运行逻辑；各游戏需提供演示步骤
插件化	预估工作量最大：把不同游戏单独打包，将通用类和具体游戏进行解耦

1.2 实际修改内容

实际代码变更比预估更彻底，不仅做了增量开发，还进行了框架级重构：

1.2.1 包结构完全重构

- **Lab4**：`com.othello.*`，所有游戏混放在 `model` 包下，命名带有 Othello 专属色彩。
- **Lab5**：`com.gameplatform.*`，按游戏类别进行分层（`model/view/controller/text/plugin/games/*`），修改原 Othello 专属命名，体现通用游戏平台的定位。

1.2.2 核心抽象层重写

- **Lab4**：`BaseGame` 以 `GameStatusCode` 枚举驱动状态机，各游戏通过 `play(Position)` 统一入口落子；`GameController` 需硬编码判断 `instanceof OthelloGame/Minesweeper` 来处理 `pass` 和 `flag`。

- **Lab5**：`BaseGame` 改为**命令驱动**，核心入口变为 `processCommand(String)`，由子类自行解析命令；状态消息改为字符串+错误标志的宽松模型。`GameController` 彻底解耦，不再关心具体游戏类型，所有游戏指令均传输给 `game.*.processCommand(cmd)`，由具体游戏进行实现。

1.2.3 界面渲染通用化

- **Lab4**：`TerminalUI.drawStatusInBox` 中充斥 `if (isReversi) ... else if (isMinesweeper) ...` 的硬编码分支，新增游戏必须改 UI 代码。
- **Lab5**：引入 `TurnBasedGame`（回合信息）和 `HintProvider`（是否需要合法落子提示）接口，UI 通过接口获取信息并通用渲染。新增游戏**无需修改 UI 代码**，只需要根据游戏类别接入接口，即可自动适配左-中-右布局。

1.2.4 多局管理插件化

- **Lab4**：`MultiBoardManager` 构造函数直接 `new PeaceGame(size)`、`new ReversiGame(size)`、`new Minesweeper(size)`，新增游戏必须修改此类。
- **Lab5**：引入 `GamePlugin` 接口和 `GamePluginRegistry` 注册表。`MultiBoardManager` 仅依赖注册表，通过 `registry.listDefaultTypeIds()` 动态创建默认对局，通过 `registry.get(type)` 动态新增对局。各游戏以独立 Plugin 类接入，**完全符合开闭原则**。

1.2.5 国际象棋完整实现

- 不仅实现了基本走子，还完整实现了
  - **王车易位**（含王/车移动标记、路径空方格检测、王经过/到达格不被将军检测）
  - **吃过路兵**
  - **兵升变**（支持升变为后/车/象/马，通过 `m e7 e8 q` 指定）
  - **将军/应将检测**（模拟走子后检测己方王是否被将军）

1.2.6 Demo 模式

- **Lab4**：无此功能。
- **Lab5**：`BaseGame` 新增 `getDemoSteps()` 与 `getDemoStepDelayMs()`；`GameController` 新增完整 Demo 状态机（`startDemo/tickDemo/abortDemo`），支持逐游戏自动演示、步骤讲解、任意键中断。

二、相对 Lab4 的增量设计点

2.1 架构层面：从硬编码到插件化

Lab4 的架构本质上是"白盒复用"：新增一种游戏必须修改 `GameController`、`MultiBoardManager` 构造函数、`TerminalUI` 三处核心代码甚至更多，违反开闭原则。

Lab5 改为"黑盒插件"：

1. **GamePlugin 接口**：约定 `getTypeId()` / `getDisplayName()` / `createGame(int)` / `isDefaultGame()` 四个方法，要求具体游戏类实现；
2. **GamePluginRegistry**：维护 `Map<String, GamePlugin>`，提供注册与查询；
3. **Main 入口**：使用 `ServiceLoader.load(GamePlugin.class)` 自动发现插件（预留 SPI 扩展点），也可手动 `register()`；
4. **各游戏独立包**：`games.chess`、`games.othello`、`games.minesweeper` 各自内聚，外部仅通过 Plugin 类暴露。

增量价值：新增第 5 种游戏时，只需新建一个包、实现 `GamePlugin` 并放入 classpath，核心代码零修改。

2.2 交互层面：从统一落子到命令传输

Lab4 中所有游戏被强制塞进 `play(Position)` 模型，导致：

- 扫雷需要 `flag(Position)` 额外方法；
- 黑白棋需要 `pass()` 额外方法；
- 国际象棋需要 `from/to` 两个坐标，无法兼容 `play(Position)`。

Lab5 改为 `processCommand(String)`：

- 国际象棋自行解析 `m e2 e4 q`；
- 扫雷自行解析 `f a1` 和 `a1`；
- 黑白棋自行解析 `pass` 和 `a1`；
- 平台层只负责识别全局命令（`quit/demo/switch/add`），其余全部透传。

```
// Lab5: ReversiGame.java — processCommand 实现
@Override
public boolean processCommand(String cmd) {
    if (cmd.equalsIgnoreCase("pass")) {
        pass();
        return true;
    }
    return super.processCommand(cmd);  // 交由 OthelloGame 处理落子
}
```

增量价值：交互模型与游戏复杂度解耦，支持任意复杂的游戏指令。

2.3 界面层面：从类型分支到接口驱动

Lab4 的 UI 代码使用 `if` 语句判断具体游戏：

```
if (isReversi) { ... } else if (isMinesweeper) { ... }
```

Lab5 引入两个细粒度接口：

- `TurnBasedGame`：提供 `getPlayerCount()` / `getCurrentPlayerIndex()` / `getPlayerName(int)`，UI 据此渲染回合高亮；
- `HintProvider`：提供 `showLegalMovesAsHints()` / `getLegalMoves()`，UI 据此在棋盘上加 `+` 提示。

增量价值：UI 代码量下降，新增游戏只要实现接口，便可自动获得回合高亮和合法落子提示能力。

2.4 功能层面：新增 Demo 模式与国际象棋

- **Demo 模式**：零人工干预自动展示游戏玩法，每个游戏通过 `DemoStep` 记录（命令+讲解文本）平台统一调度；
- **国际象棋**：完整规则引擎，作为验证平台可扩展性的测试。

```
// Lab5: ChessGame.java
@Override
public List<DemoStep> getDemoSteps() {
    return List.of(
        new DemoStep("m e7 e5", "【兵的双步】白方e兵前进两格——兵第一步可走两格"),
        new DemoStep("m e2 e4", "黑方e兵对称应对"),
        new DemoStep("m g8 f6", "【马的走法】白马跳f6——马走\"日\"字，可跳过棋子"),
        new DemoStep("m b1 c3", "黑马跳c3——对称出子"),
        new DemoStep("m f8 c5", "【象的走法】白象斜走c5——象沿斜线移动"),
        new DemoStep("m f1 c4", "黑象斜走c4——双方象对峙"),
        new DemoStep("m e8 g8", "【王翼易位】白方短易位0-0！王向右两格，车跨到王旁——王车易位"),
        new DemoStep("m g1 f3", "黑马跳f3——中局出动"),
        new DemoStep("m b8 c6", "【马的走法】白后翼马跳c6——两匹马均已出动"),
        new DemoStep("m e1 g1", "【王翼易位】黑方短易位——双方均完成王车易位"),
        new DemoStep("m d7 d6", "白d兵走一步——相机巩固中心"),
        new DemoStep("m d2 d3", "黑d兵走一步——为后翼象开路"),
        new DemoStep("m c8 g4", "【象远程牵制】白象远程斜走g4——牵制黑马"),
        new DemoStep("m h2 h3", "黑h兵走一步——驱赶白象"),
        new DemoStep("m g4 f3", "【象吃马】白象吃黑马——象斜线吃子"),
        new DemoStep("m d1 f3", "黑后吃白象——后吃子"),
        new DemoStep("m c6 d4", "【马攻后】白马跳d4威胁黑后——马走日字制造威胁"),
        new DemoStep("m f3 d1", "黑后撤回d1——后沿斜线撤退"),
        new DemoStep("m d4 e2", "【马将军】白马跳e2将军！——马的将军"),
        new DemoStep("m g1 h1", "【王的走法】黑王逃到h1——王每次只能走一格"),
        new DemoStep("m e2 c1", "【马吃象】白马吃黑象——马吃子"),
        new DemoStep("m a1 c1", "【车吃马】黑车横走吃白马——车沿横线吃子"),
        new DemoStep("m a7 a5", "白a边兵推进——准备吃过路兵演示"),
        new DemoStep("m a2 a3", "黑a兵走一步"),
```

```
            new DemoStep("m a5 a4", "白兵推进到a4——到达第4横线"),
            new DemoStep("m b2 b4", "黑b兵双步跳到b4——创造过路兵机会！"),
            new DemoStep("m a4 b3", "【吃过路兵!】白a4兵斜吃b4过路兵到b3——过路兵特殊规则"),
            new DemoStep("m g2 g4", "黑g兵双步推进"),
            new DemoStep("m b3 b2", "白兵推进到b2——逼近底线，即将升变！"),
            new DemoStep("m g4 g5", "黑g兵推进一步"),
            new DemoStep("m b2 c1 q", "【兵升变为后!】白兵斜吃黑车冲到底线，升变为后——兵到底线可变后/车/象/马")
        );
    }
```

三、新增/修改的关键类与职责

3.1 平台核心（修改/新增）

类	状态	职责说明
`BaseGame`	**重写**	移除 `GameStatusCode` 状态机；新增 `processCommand()`、`getExtraStatusLines()`、`getAvailableCommands()`、`getDemoSteps()`
`TurnBasedGame`	**新增接口**	抽象回合制游戏的玩家轮询信息，供 UI 通用渲染
`HintProvider`	**新增接口**	抽象可提示合法落子的游戏能力，供棋盘 `+` 号提示
`MultiBoardManager`	**重写**	移除硬编码 `new XXXGame()`，改为依赖 `GamePluginRegistry`；新增 `resetToDefaults()`、`isSupportedType()`、`getAddableTypes()`
`GameController`	**重写**	移除所有 `instanceof` 分支和 `parsePosition` 逻辑；新增 Demo 状态机（`demoMode`、`tickDemo()`）；命令处理简化为平台命令+透传
`TerminalUI`	**重写**	移除所有游戏类型硬编码分支；通过 `TurnBasedGame` / `HintProvider` 接口通用绘制状态面板和棋盘提示
`GameTexts`	**重写**	移除 Lab4 中大量游戏专属文本方法，仅保留平台级公共文本；游戏专属文本下放到各游戏包

3.2 插件层（新增）

类	职责
`GamePlugin`	插件契约：类型 ID、显示名、创建游戏实例、是否默认游戏
`GamePluginRegistry`	插件注册表，维护 `Map<String, GamePlugin>`，支持按类型 ID 查询
`PeacePlugin`	和平棋插件适配器
`ReversiPlugin`	黑白棋插件适配器
`MinesweeperPlugin`	扫雷插件适配器

类	职责
`ChessPlugin`	国际象棋插件适配器

3.3 国际象棋模块（新增）

类	职责
`ChessGame`	核心游戏引擎：行棋规则验证、特殊规则（易位/过路兵/升变）将军检测、胜负判定、Demo 步骤编排
`ChessBoard`	8×8 棋盘数据：标准开局布子、棋子存取、越界检测、深拷贝
`ChessPiece`	棋子枚举：含颜色、兵种判断（`isKing()` / `isPawn()` 等）、显示字符（大写黑/小写白）
`Color`	颜色枚举：BLACK / WHITE / NONE
`ChessTexts`	国际象棋专属文本：错误提示、命令格式、胜负结果
`MoveCandidate`	内部数据类：记录一次候选走子的完整信息（起点、终点、被吃子、是否升变/易位/过路…
`BoardState`	内部接口：为将军检测提供只读棋盘视图，支持真实棋盘和模拟数组两种实现

3.4 既有游戏迁移（修改）

类	修改说明
`PeaceGame`	继承新版 `OthelloGame`；移除 `GameStatusCode`；改为字符串状态；新增 `getAvailableCommands()` / `getDemoSteps()`
`ReversiGame`	同上；实现 `HintProvider`；`pass()` 逻辑内聚到本类；新增 `getDemoSteps()`
`Minesweeper`	抽离 `MineField` 负责地雷与格子状态；`Minesweeper` 负责命令解析与胜负判定；新增固定雷场供 Demo 使用
`MineField`	**新增**（从原 Minesweeper 抽离）：专注地雷生成、格子翻开、插旗、邻雷计数

四、国际象棋规则说明

4.1 各棋类行棋规则

棋子	走法	实现要点
王 (K/k)	周围 8 格的其中一格	`ChessGame.generateCandidates()` 中遍历 8 邻域；附加易位逻辑
后 (Q/q)	横/竖/斜线任意格	复合车＋象方向，逐格检测直到撞子

棋子	走法	实现要点
车 (R/r)	横/竖线任意格	四个正方向逐格检测
象 (B/b)	斜线任意格	四个对角方向逐格检测
马 (N/n)	"日"字，可越子	8 个偏移量直接跳转，不检测路径障碍
兵 (P/p)	直走一格（首步可两格）、斜吃一格、只能向前	方向由颜色决定（白向上/黑向下），首步双步记录到 `lastMove` 用于过路兵

4.2 特殊规则实现

4.2.1 王车易位

- **前提条件**：王和对应车均未移动过（`whiteKingMoved` / `whiteLeftRookMoved` 等布尔标记）。
- **空方格检测**：王与车之间所有格子必须为空；短易位检测 f/g 列，长易位额外检测 b 列（因车需经过 d 列到达 c 列旁）。
- **安全检测**：王当前位置、经过的格子、到达的格子均不能被对方将军（`isSquareAttacked`）。
- **执行**：王移动两格，车跳到王相邻内侧；更新对应车的移动标记。

4.2.2 吃过路兵

- **触发条件**：对方兵首步直走两格，且紧邻我方兵。
- **记录**：`lastMove.doublePawnStep` 标记上一步是否双步兵；`lastMove.to` 记录落点。
- **执行**：我方兵斜行至对方兵相邻空位，同时移除对方双步兵（通过 `lastMove.to` 定位）。

4.2.3 兵升变

- **触发**：兵到达对方底线。
- **命令格式**：`m e7 e8 q`（可选第 4 参数 `q/r/b/n`，默认后）。
- **实现**：`getPromotionPiece(char, boolean)` 映射字符到对应颜色的棋件；若命令显式指定升变类型但走法本身非升变，则报错 `PROMOTION_NOT_ALLOWED`。

4.3 胜负判定

采用**简化规则**：当一方王棋被吃掉时，游戏立即结束。

- 实现位于 `playWithPromotion()` 末尾：执行走子后，若 `selected.captured != null && selected.captured.isKing()`，则调用 `setOver()` 并显示 `KING_CAPTURED`。
- 结束后 `processCommand()` 对任何 `move` 命令均返回 `GAME_OVER` 错误提示。

- **注意**：实际代码中同时实现了**将军检测**（`isKingSafeAfterMove`），非法的"送王"走子会被拒绝并提示 `KING_IN_CHECK`，这保证了王不会被"意外"吃掉，只有在对方合法将军且用户未应将时才会被吃。

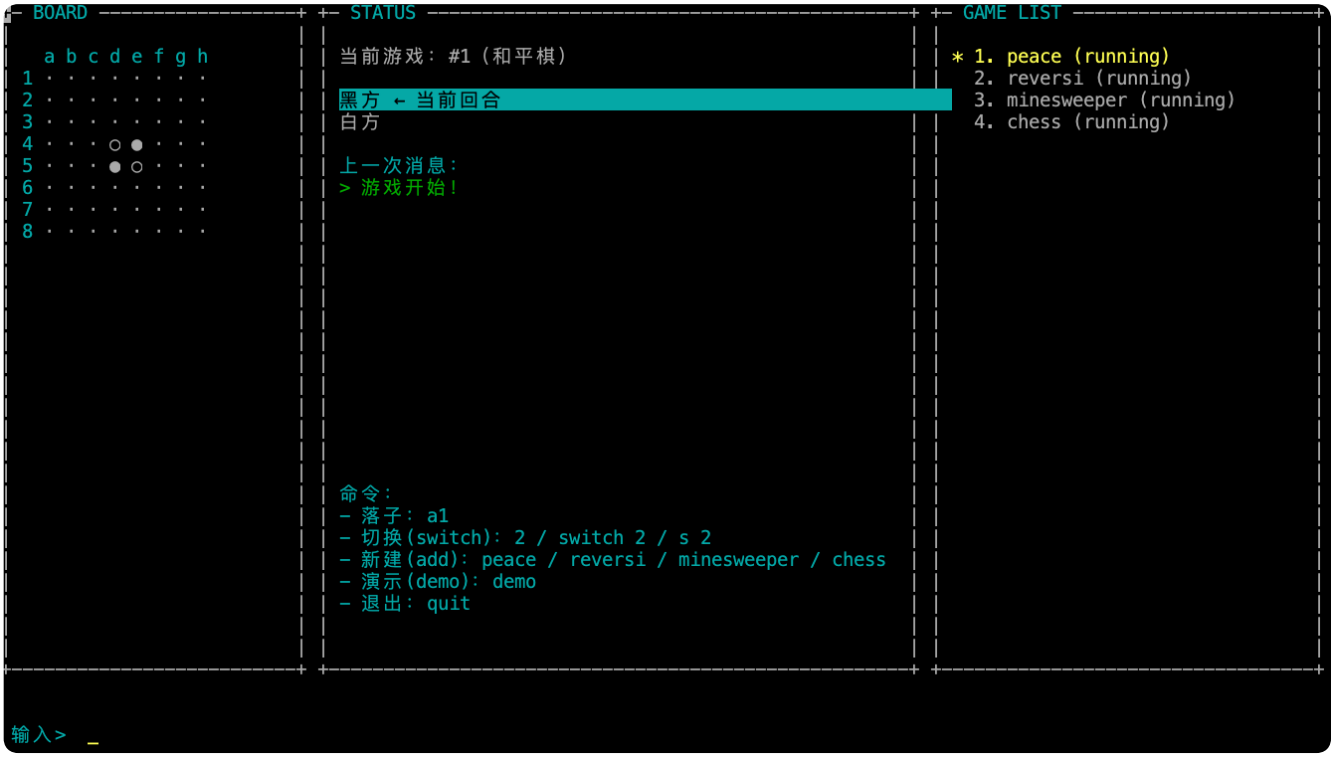
五、多对局与动态新增流程截图展示

5.1 初始状态

程序启动后，`MultiBoardManager` 通过 `GamePluginRegistry` 加载所有 `isDefaultGame() == true` 的插件，按注册顺序生成 4 局：

1. `1. peace`（和平棋）
2. `2. reversi`（黑白棋）
3. `3. minesweeper`（扫雷）
4. `4. chess`（国际象棋）

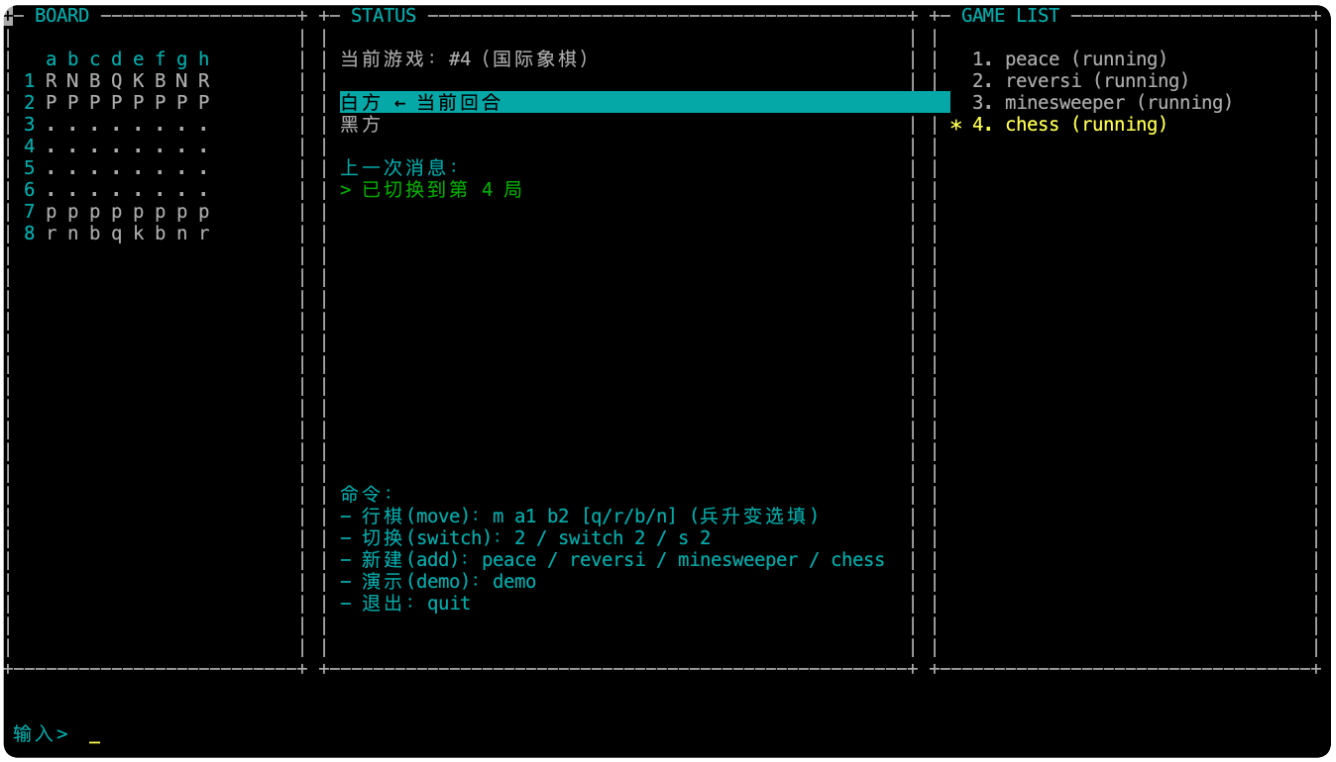
默认当前对局为 `1.`。



5.2 切换对局

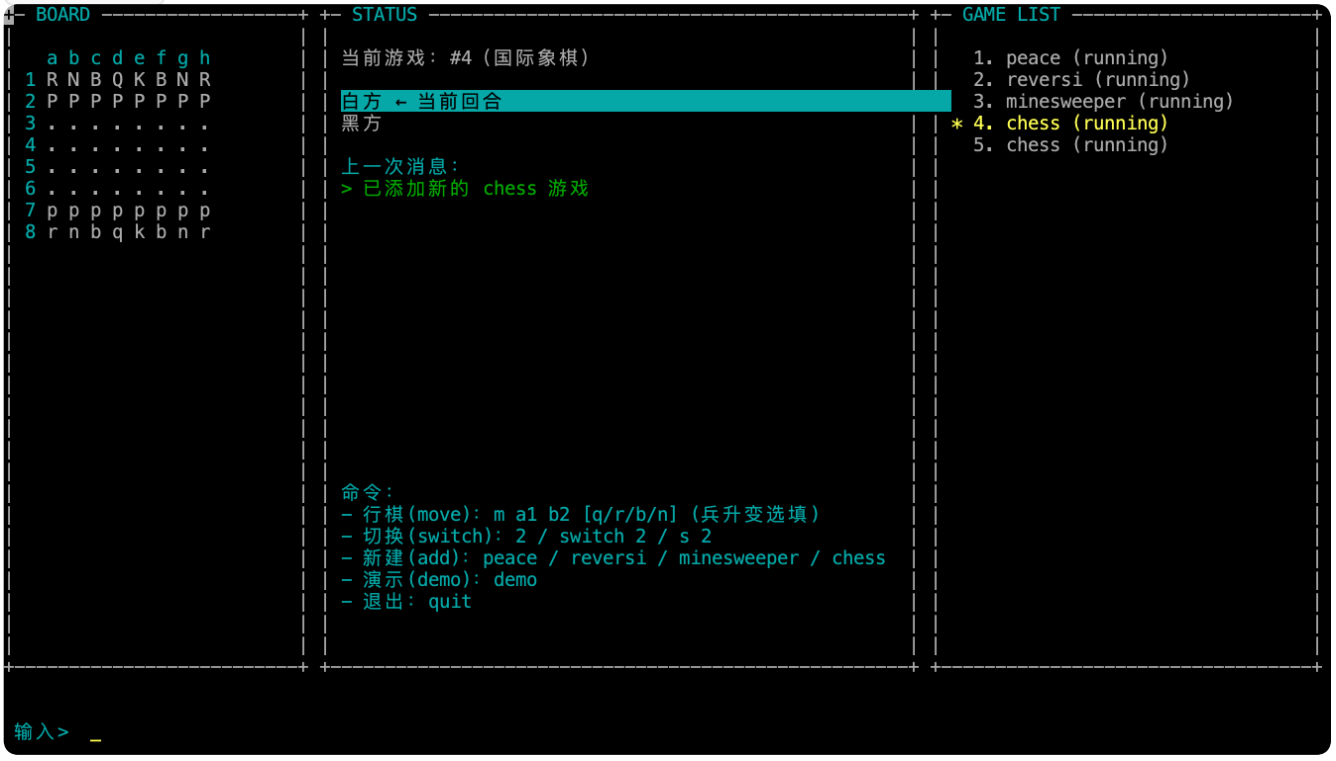
输入 `4` 或 `s 4` 或 `switch 4`，`GameController.parseBoardNumber()` 解析编号，`MultiBoardManager.select(4)` 切换索引。由于每局 `BaseGame` 独立持有状态，切换后原对

局状态完整保留。



5.3 动态新增

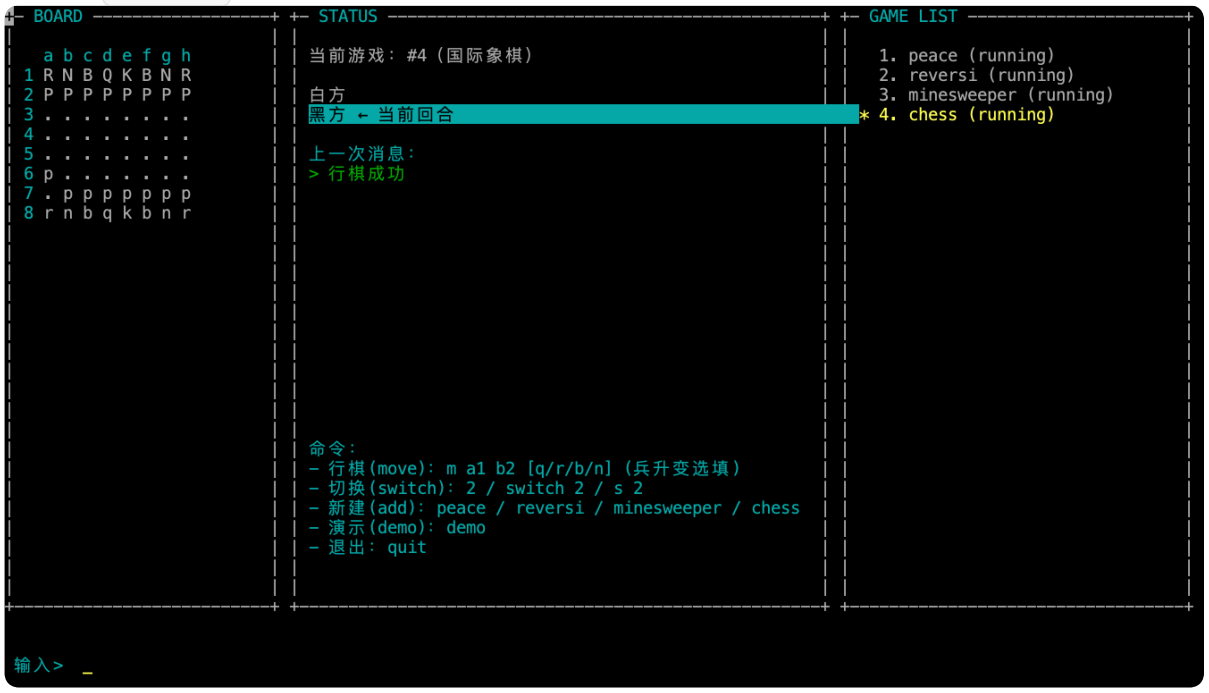
输入 `chess`，`GameController` 检测到 `manager.isSupportedType("chess")` 为 true，调用 `manager.add("chess", 8)`，注册表返回 `ChessPlugin` 创建新实例，追加到列表末尾。右侧 `GAME LIST` 实时刷新显示新对局。



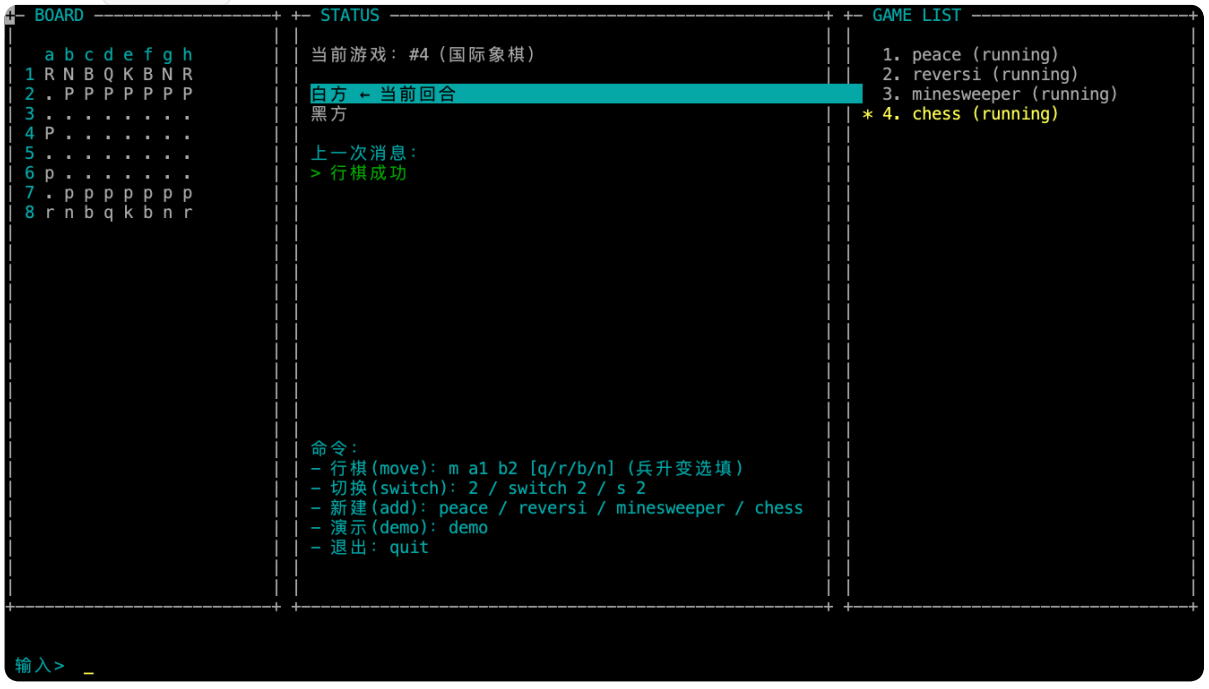
六、国际象棋截图展示

6.1 棋子的基本走法

- 兵
  - 输入： `m a7 a6`

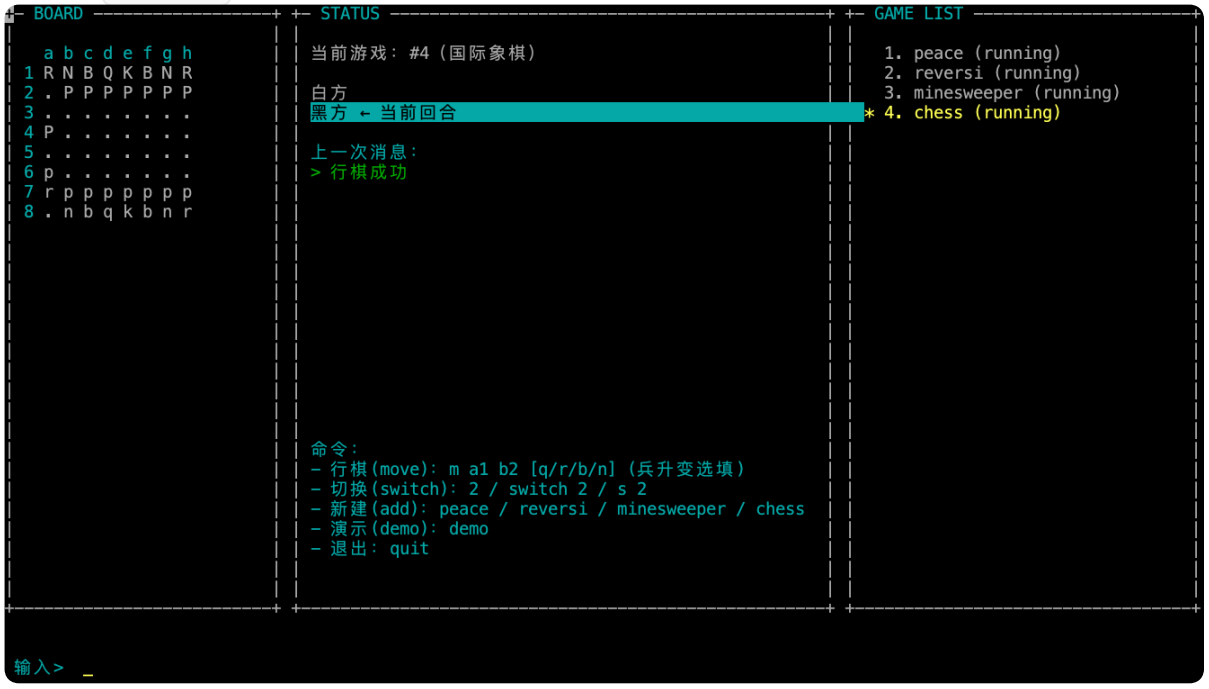


  - 输入： `m a2 a4`

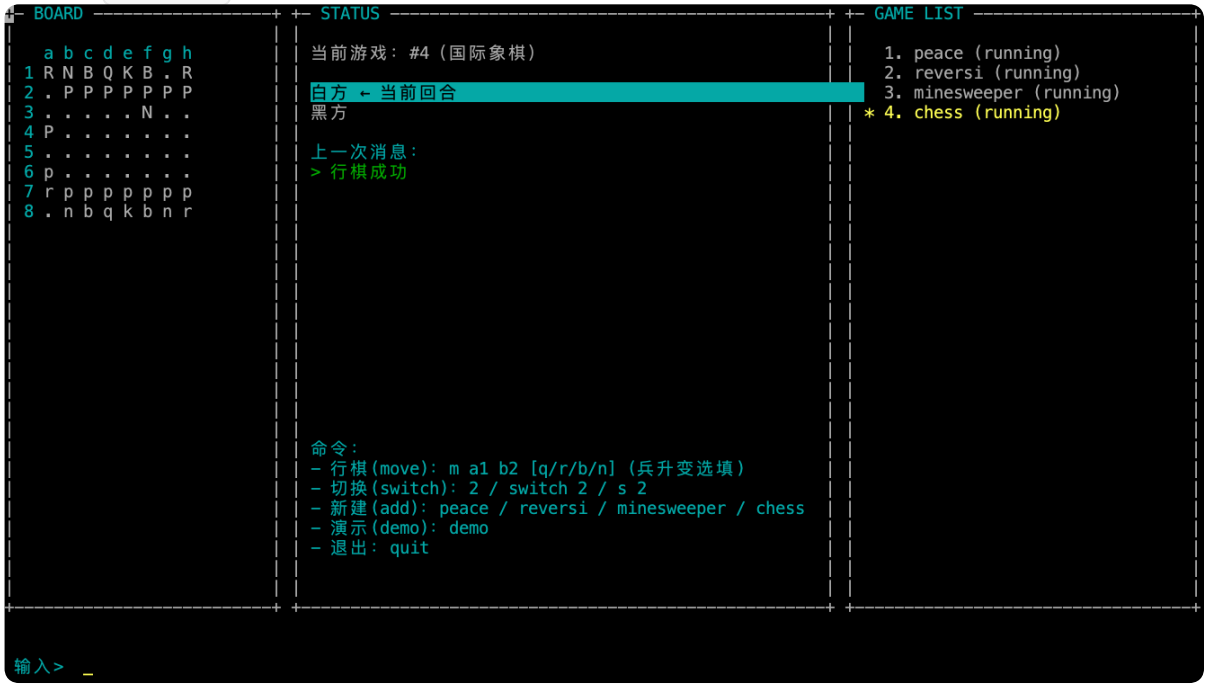


- 车

- 输入：`m a8 a7`

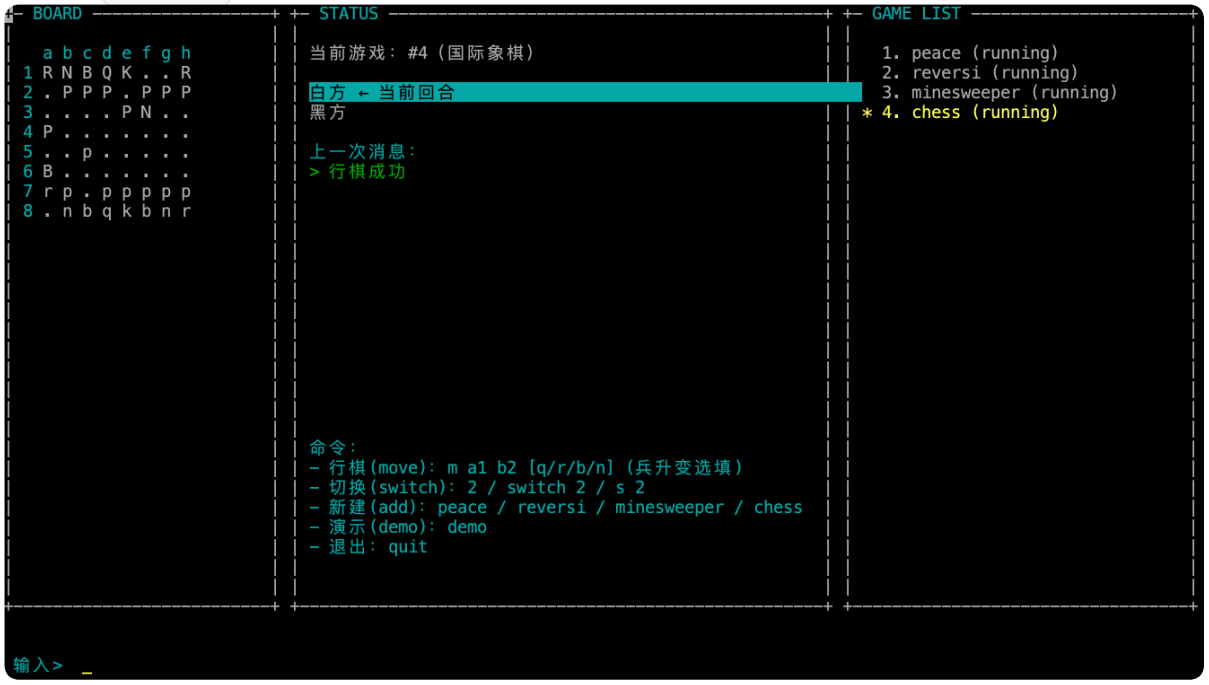


- 马
  - 输入：`m g1 f3`

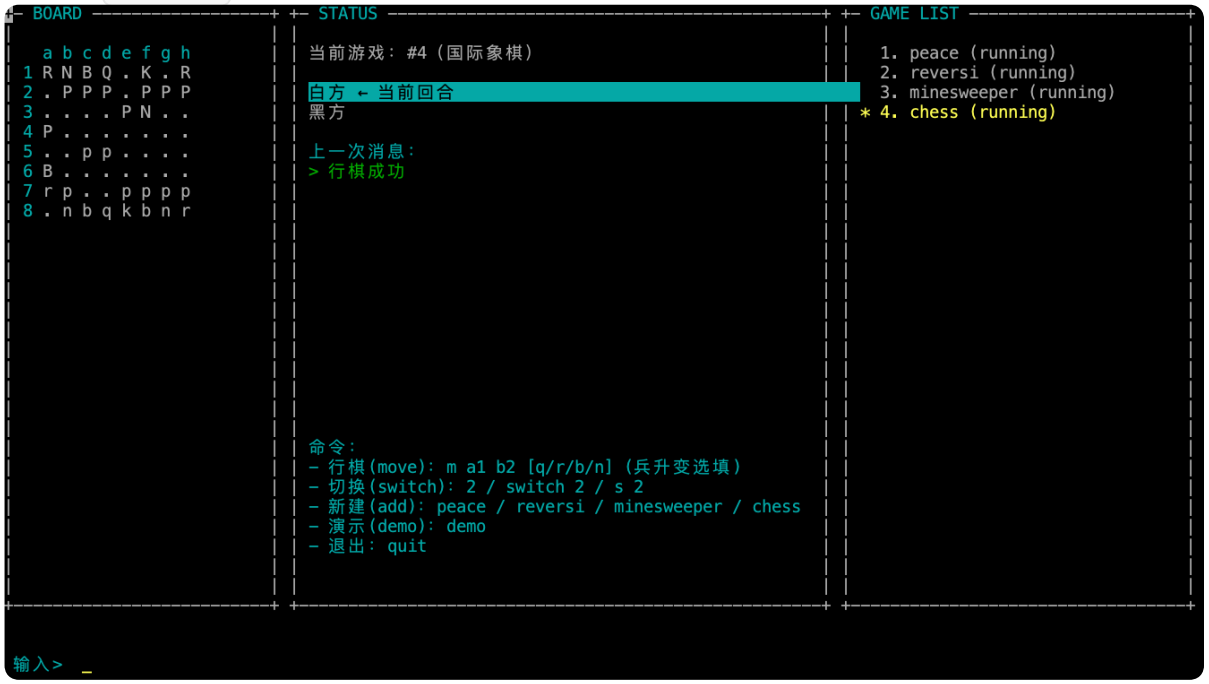


- 象

- 输入：`m f1 a6` （吃子）

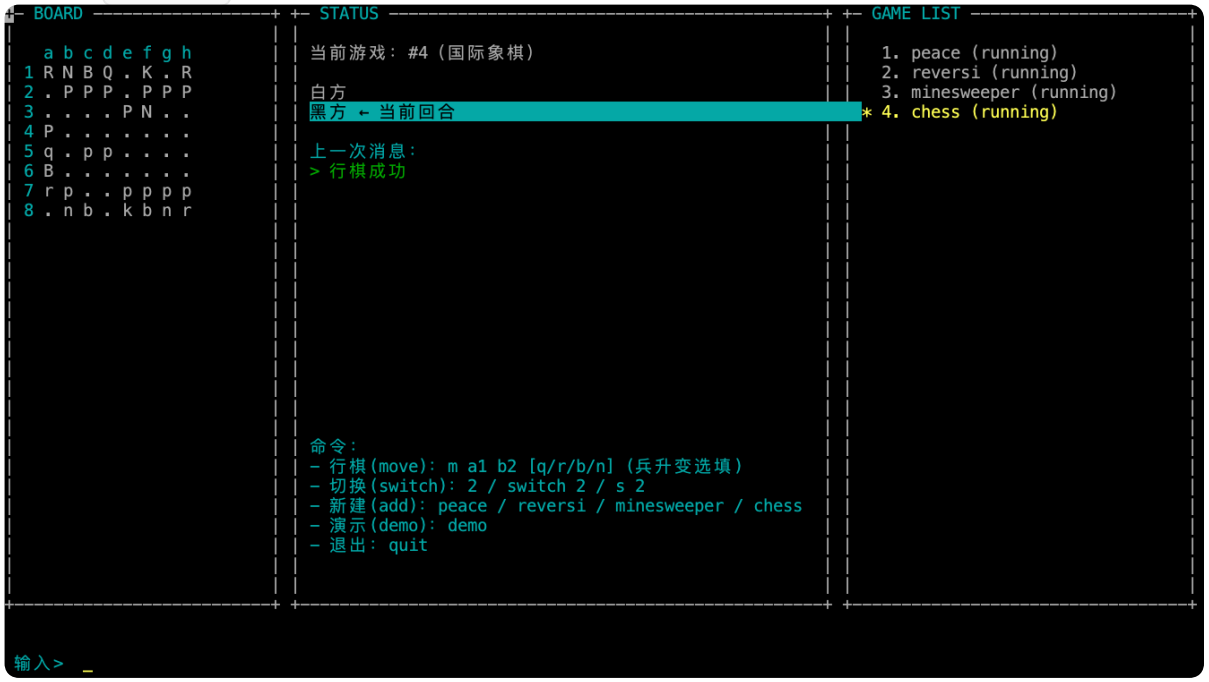


- 王
  - 输入：`m e1 f1`



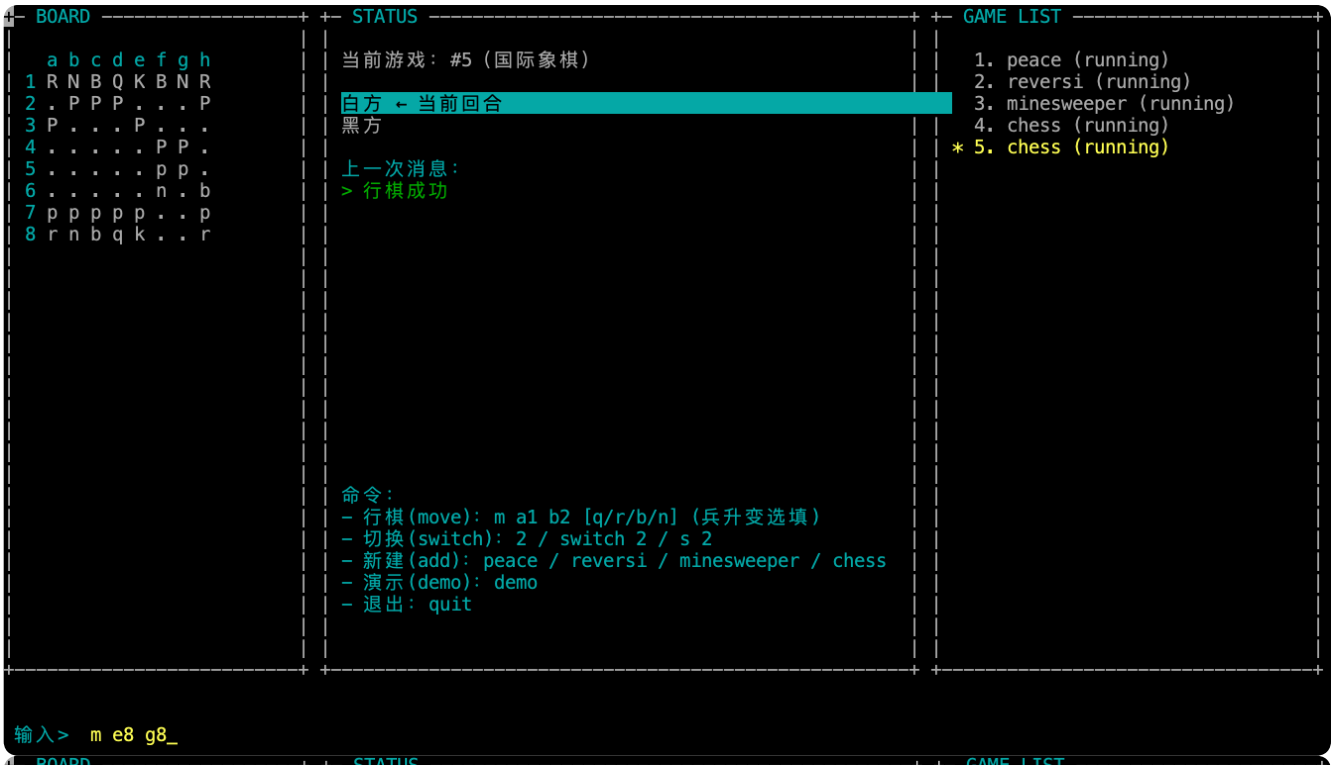
- 后

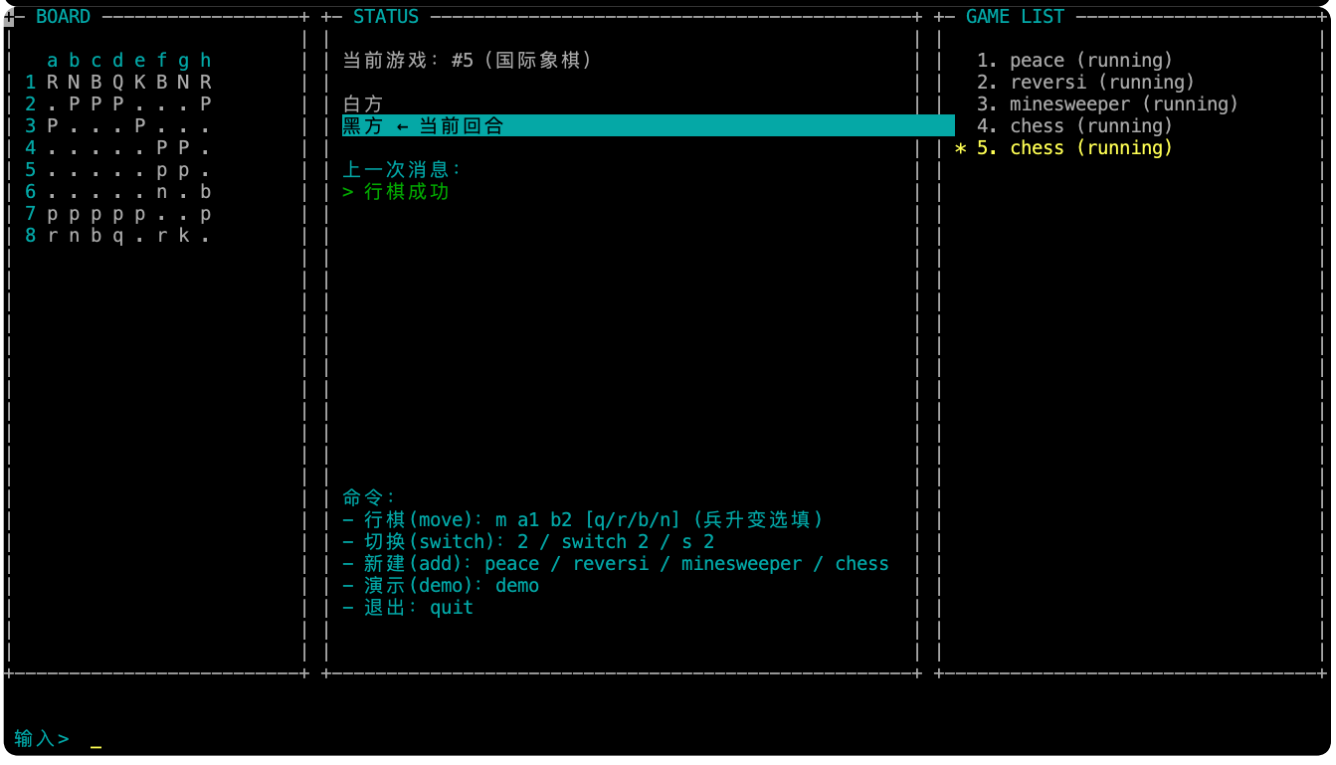
- 输入： `m d8 a5`



6.2 特殊规则

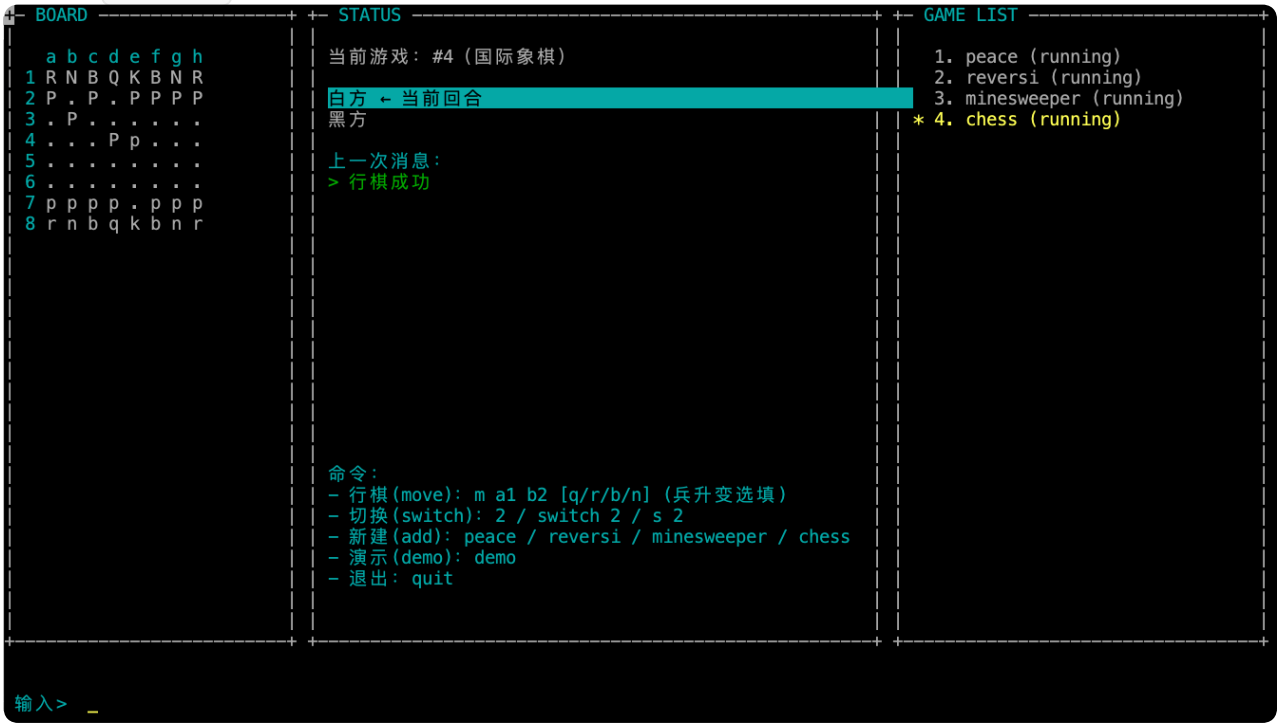
6.2.1 王车易位


BOARD
STATUS
GAME LIST
a b c d e f g h
1 R N B Q K B N R
2 . P P P . . . P
3 P . . . P . . .
4 . . . . . P P .
5 . . . . . p p .
6 . . . . . n . b
7 p p p p p . . p
8 r n b q k . . r
当前游戏: #5 (国际象棋)
白方 ← 当前回合
黑方
上一次消息:
> 行棋成功
命令:
- 行棋(move): m a1 b2 [q/r/b/n] (兵升变选填)
- 切换(switch): 2 / switch 2 / s 2
- 新建(add): peace / reversi / minesweeper / chess
- 演示(demo): demo
- 退出: quit
1. peace (running)
2. reversi (running)
3. minesweeper (running)
4. chess (running)
* 5. chess (running)
输入> m e8 g8_

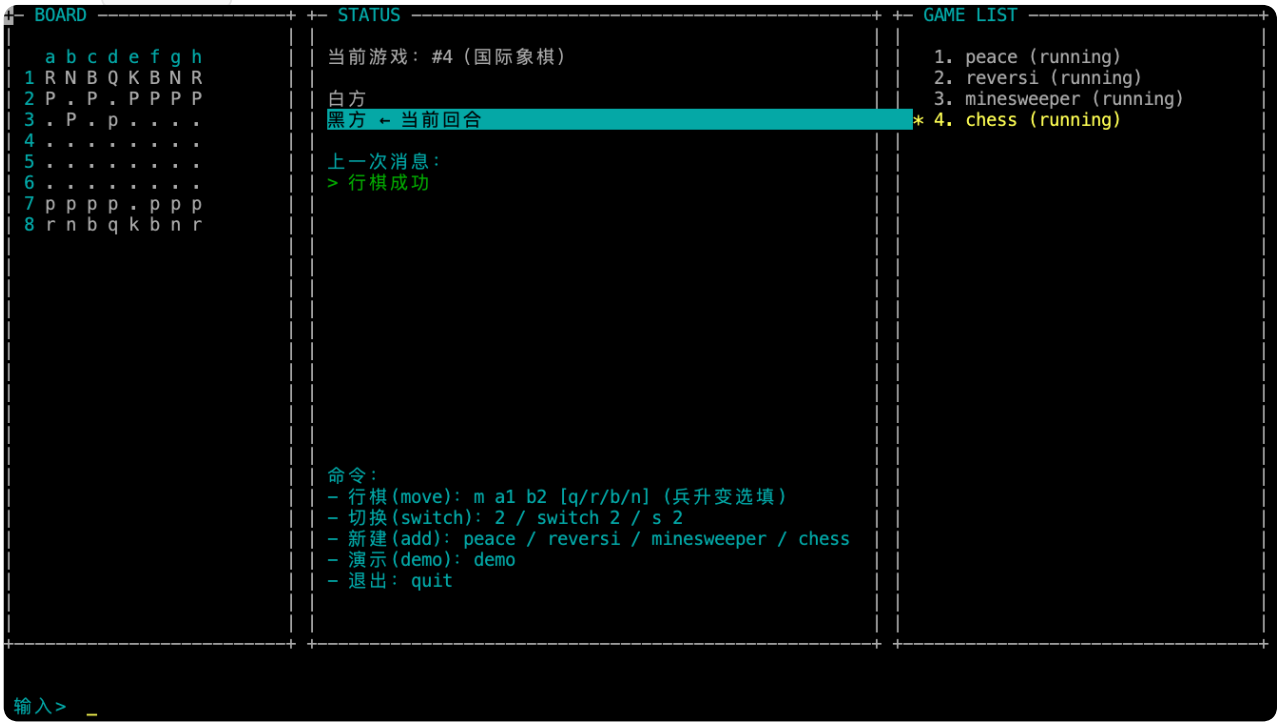

BOARD
STATUS
GAME LIST
a b c d e f g h
1 R N B Q K B N R
2 . P P P . . . P
3 P . . . P . . .
4 . . . . . P P .
5 . . . . . p p .
6 . . . . . n . b
7 p p p p p . . p
8 r n b q . r k .
当前游戏: #5 (国际象棋)
白方
黑方 ← 当前回合
上一次消息:
> 行棋成功
命令:
- 行棋(move): m a1 b2 [q/r/b/n] (兵升变选填)
- 切换(switch): 2 / switch 2 / s 2
- 新建(add): peace / reversi / minesweeper / chess
- 演示(demo): demo
- 退出: quit
1. peace (running)
2. reversi (running)
3. minesweeper (running)
4. chess (running)
* 5. chess (running)
输入> _

6.2.2 吃过路兵

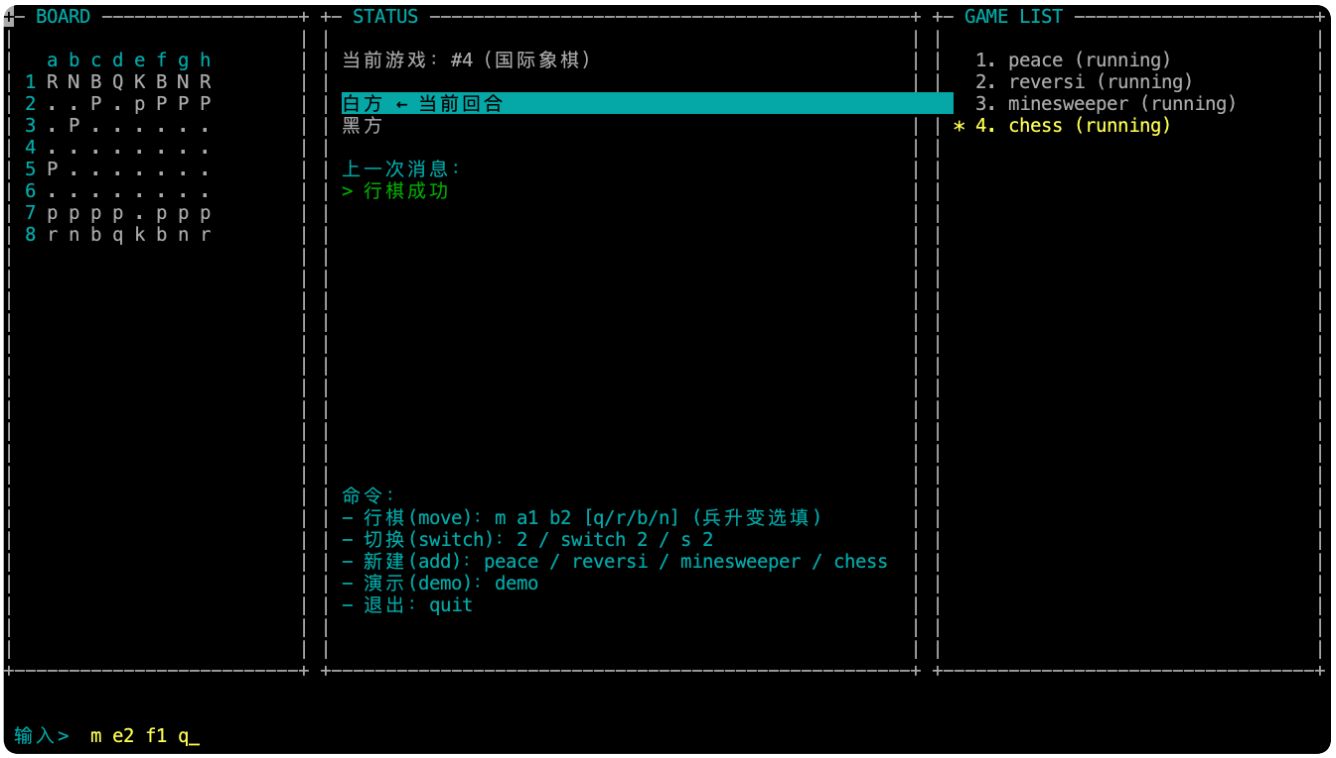
- 输入： `m d2 d4`

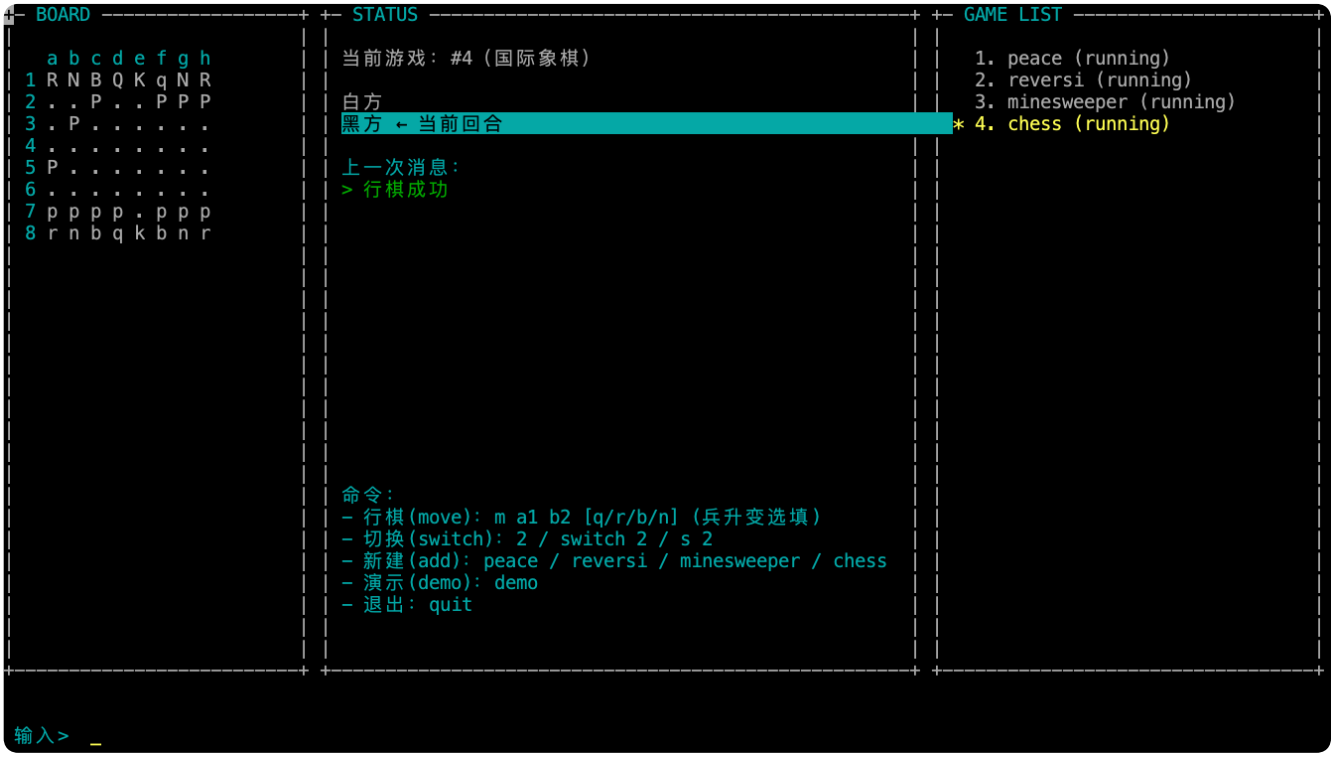


- 输入： `m e4 d3`



6.2.3 兵的升变





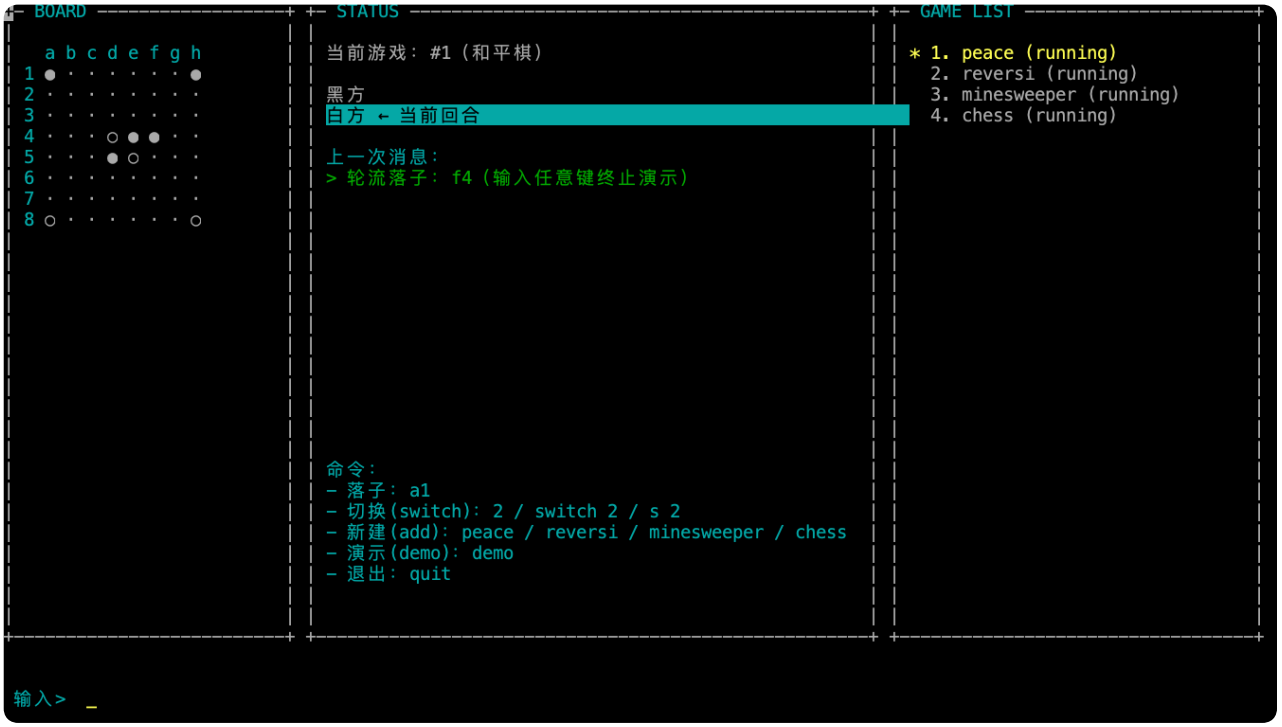
七、Demo 模式流程截图展示

7.1 启动

输入 `demo`，`GameController.startDemo()` 被触发：

- 重置 `demoMode = true`，`demoGameIndex = 0`，`demoStepIndex = -1`；
- 选中第 1 局，显示 "即将开始演示:「和平棋」 ";

- 设置 1500ms 延迟后执行第一步。



7.2 逐游戏演示

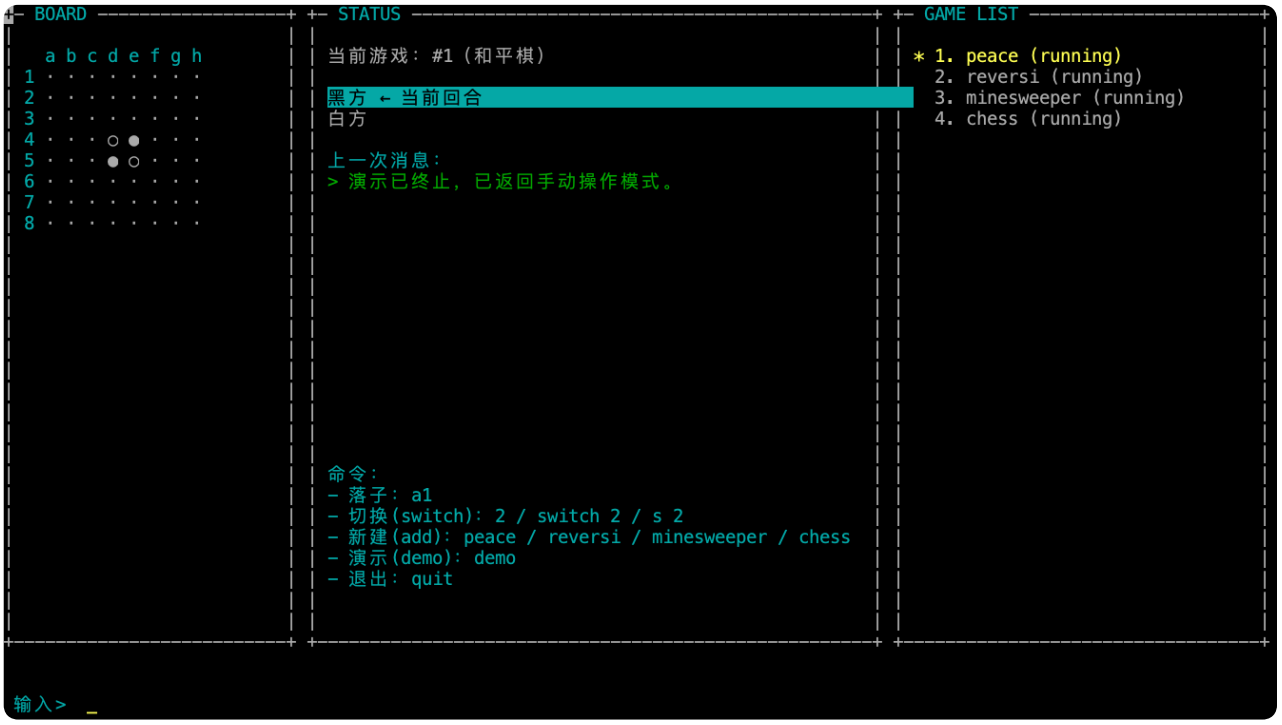
`tickDemo()` 按以下循环调度：

1. **游戏间切换**：当前游戏步骤耗尽后，`demoGameIndex++`，选中下一局，显示切换提示（如"演示完成:「和平棋」 。切换到下一个游戏:「黑白棋」"） 。
2. **步骤执行**：每步取出 `DemoStep(command, explanation)`，将 `explanation` 显示为平台消息，执行 `command` 驱动游戏状态变更。
3. **延迟控制**：每步间隔由 `BaseGame.getDemoStepDelayMs()` 决定（和平棋 800ms / 扫雷 1000ms / 黑白棋 1500ms / 国际象棋 1200ms）。



7.3 中断与结束

- **任意键中断**：Demo 运行期间，`GameController` 通过 `ui.pollKey()` 检测按键，一旦检测到输入，立即调用 `abortDemo()`，重置所有对局到初始状态，返回手动模式。
- **自然结束**：所有游戏演示完毕后，`demoMode = false`，显示 `DEMO_FINISHED`，重置默认 4 局。



7.4 各游戏 Demo 内容设计

游戏	演示重点
Peace	四角+中间轮流落子，展示回合交替机制
Reversi	夹吃翻转、占角策略、Pass 跳过规则
Minesweeper	首开安全、数字推理、插旗/取消插旗
Chess	兵双步、马走日、象斜走、王车易位、吃过路兵、兵升变

八、结论

本次 Lab5 在 Lab4 的终端游戏平台基础上，完成了两项核心任务：

1. **功能增量**：新增国际象棋完整规则引擎与 Demo 自动演示模式，验证平台对复杂回合制游戏的支持能力；
2. **架构升级**：通过插件化重构（GamePlugin + Registry + 接口驱动 UI），将 Lab4 中"新增游戏必改三处核心代码"的白盒架构，升级为"新增游戏只增一个包、零改核心代码"的黑盒架构。

与 Lab4 相比，代码耦合度显著降低，各游戏模块内聚度提升，UI 渲染完全通用化，为后续继续扩展奠定了可插拔的框架基础。